Harness² runtime comparison: OpenCode vs Codex vs Grok Build vs Pi

Date: 2026-11-05 **Task:** `feature_pipeline-hard-window_aggregates` (WorkBuddy-Bench code subset, 13 hidden tests) **Runs:** one pipeline run per runtime (`k=1`)

1. Summary

Runtime	Before	After	Change	Final still fails	Improver time per session
OpenCode	11/13	**13/13**	**+2**	none	21 to 95 min (2 at a time, 3 tasks queued)
Codex	**13/13**	**13/13**	0 (already perfect)	none	15 to 34 min
Grok Build	11/13	11/13	0	`last event hours u1/u2`	16 to 53 min
Pi	5/13	11/13	**+6**	`last event hours u1/u2`	8 to 11 min

- **Best final score:** OpenCode and Codex (13/13). Codex reached it without help from the improver.
- **Largest gain from the improver:** Pi (+6), starting from the weakest base.
- **Fastest improver loop:** Pi, then Codex.
- **Weakest result:** Grok. One of its candidates reached 13/13, but the composition step chose the other candidate.

One run per runtime is not proof. On other tasks, the base score varied by 1 to 2 tests between identical runs. Treat the order as a first indication.

2. Common setup (identical for all four)

Role	Model	Where it runs
Executor (solver)	`sofie-code-0`	Inside the task's Docker container, in the runtime being tested
Improver (INV, DOM, AGG)	`sofie-think-0`	In the same runtime as the executor, in its own container (OpenCode on the host)
Grader	Hidden tests (13)	Task container; the improver never sees them

- **Endpoint:** university server `llm.karunya.edu`. Codex, Grok and Pi reach it through a local LiteLLM bridge on port 45057. OpenCode connects directly.
- **Method:** Google's design, with no test feedback to the improver. Per task: base run, INV and DOM improver sessions, one test run per candidate, AGG composes the final harness, then the final run.

- **Harness files the improver edits:** `systemprompt.md`, guardrails and skills for OpenCode. `AGENTS.md`, `.agents/skills/` and `scripts/` for Codex, Grok and Pi. All three read that layout, which I confirmed with a check on each tool.

3. The task

The request is in Chinese. In translation: fix the windowed user-event features so that only events before `anchor_time` count. Compute 1-day and 7-day counts, amounts and hours since the last event, and exclude events exactly at `anchor_time`. The reference change is 1 file and 32 lines. The task is rated *hard*. In a 16-task screen, it was one that the base agent nearly solved.

4. OpenCode

Stage	Harness change	Tests	Tool calls
Base	none	11/13	20
INV candidate	skill `verify-window-aggregates` + system-prompt trigger	6/13	15
DOM candidate	skill `window-aggregates` + guardrail	3/13	11
Final	merged skill `window-aggregates` + system-prompt trigger	**13/13**	20

- **Why it improved:** the base used `int(seconds / 3600)` and failed `last event hours u1/u2`. The final skill said “exact fractional hours, never `int()`”, and the final agent wrote `.total_seconds() / 3600`.
- **Notes:** OpenCode is the method’s default runtime and needed no new code. Its improver sessions were the slowest, because the pipeline allows only 2 at a time and this run had 3 tasks queued.

5. Codex

Stage	Harness change	Tests	Tool calls
Base	none	**13/13**	20
INV candidate	`AGENTS.md` edit	5/13	15
DOM candidate	skill `window-aggregates` + `AGENTS.md` edit	5/13	21
Final	trimmed `AGENTS.md` edit	**13/13**	13

- **Result:** the base already passed everything, so the improver had nothing to gain here. Both candidates hurt (5/13). The composition step kept a harness that still passed 13/13.
- **Built for this:** a Codex improver channel. Codex’s own Linux sandbox can’t start on this Ubuntu (AppArmor blocks it), so improver sessions run in a throwaway Docker container that sees only a temporary folder. The bridge was moved to `0.0.0.0` so containers can reach it.

6. Grok Build

Stage	Harness change	Tests	Tool calls
Base	none	11/13	15
INV candidate	`AGENTS.md` edit + script `check_window_features.py`	**13/13**	12
DOM candidate	skill `window-aggregates` + `AGENTS.md` edit	11/13	16
Final	DOM-style skill + `AGENTS.md`	11/13	17

- **Result:** no improvement. The INV candidate fixed the task, but the composition step can't see test scores (by design) and kept the DOM-style changes.
- **Problems along the way:** the first DOM session ran for 4 h 46 min, until the API keys expired, because no hard time limit applied. I added a 60-minute hard stop on the container. The rerun took 16 minutes.
- **Built for this:** a Grok runner (`grok -p`, output converted to the shared transcript format), a trusted-folder setting (Grok ignores `AGENTS.md` in untrusted folders), and a Grok improver channel in Docker. Grok was already configured on this laptop for `sofie-code-0` and `sofie-think-0`.

7. Pi

Stage	Harness change	Tests	Tool calls
Base	none	5/13	17
INV candidate	`AGENTS.md` edit	5/13	11
DOM candidate	skill `window-aggregation` + `AGENTS.md` edit	11/13	18
Final	DOM-style skill + `AGENTS.md`	**11/13**	22

- **Result:** +6 tests. The base failed 8 tests on window boundaries, leakage and recency. The final failed only the two `last event hours` tests, the same rounding issue as the other runtimes' bases.
- **Not confirmed:** I found no read of the skill file in the transcript. The gain probably came from the `AGENTS.md` change, which Pi always loads.
- **Built for this:** installed Pi 0.73.1 in `pi`; wrote a Pi runner (`pi -p --mode json`, with Sofie set up as an OpenAI-compatible provider in `models.json`) and a Pi improver channel in Docker with a 60-minute cap. Smoke tests confirmed Pi follows `AGENTS.md`, edits files and reaches Sofie through the bridge.

8. What the four runs have in common

1. **Candidates often hurt.** 6 of 8 candidate runs scored at or below the base. The composition step decided each outcome.
2. **The composer can't see scores.** That rescued OpenCode and Pi, and cost Grok its 13/13.
3. **The same two tests are the hardest.** `last event hours u1/u2` need exact fractional hours, and three of the four runtimes' bases missed them.

9. Limits

- One task and one run per cell. Single runs can move by 1 to 2 tests.

- The OpenCode run shared its improver queue with two other tasks, so its timing isn't directly comparable.
- Codex had no room to improve on this task.
- The Codex, Grok and Pi runners and improver channels are local patches written for this comparison. I tested each one, but they haven't been used beyond these runs.

10. Recommendation (provisional)

- **For the best score with the least effort:** Codex. It had the strongest base and kept 13/13.
- **For showing the improver working:** Pi (largest gain, fastest loop) or OpenCode (fixed the exact failing tests).
- **Not recommended for now:** Grok. It's the slowest, and its improvement was lost at composition.

To choose with confidence: repeat base and final 3 times per runtime on this task and 1 to 2 more, then compare averages. The API keys expire at 09:50 IST on 2026-11-06.

11. Changes made to the repo for this comparison

`harness2/core/proposer.py` (Codex, Grok and Pi improver channels, with hard time limits), `harness2/core/spec.py` and `harness2/config.py` (channel selection: `HARNESS2_PROPOSER_CHANNEL`), `harness2/benchmarks/workbuddy.py` (executor selection: `HARNESS2_CX_AGENT=grok|pi`, plus binary mounts), new runners `grok_harness_agent.py` and `pi_harness_agent.py` with their harness configs, runner registration in the benchmark, and a regenerated benchmark patch (the old one is kept as `.orig`).